

Lecture 7.4

谢丹
清华大学数学系

December 3, 2025

Two-Stage LLM Training Pipeline

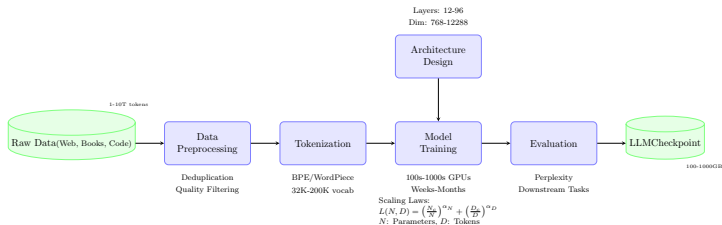
1. Pre-Training Phase

- ▶ **Objective:** Learn general language understanding and generation
- ▶ **Data:** Massive text corpora (1-10 trillion tokens)
- ▶ **Method:** Unsupervised learning via next-token prediction
- ▶ **Outcome:**
 - ▶ Base model with world knowledge
 - ▶ Text generation capability
 - ▶ Fundamental language understanding

2. Post-Training Phase

- ▶ **Supervised Fine-Tuning (SFT):**
 - ▶ Uses labeled instruction datasets
 - ▶ Aligns model with human preferences
 - ▶ Enables task-specific behaviors
- ▶ **Reinforcement Learning (RLHF):**
 - ▶ Further refines model outputs
 - ▶ Uses human/AI feedback signals
 - ▶ Optimizes for helpfulness/safety
- ▶ **Reinforcement Learning to gain reasoning capabilities:**

LLM Pre-Training Pipeline



Step 1: Data Collection

Key Data Sources

- ▶ Web pages (Common Crawl, Wikipedia)
- ▶ Books (Project Gutenberg, proprietary collections)
- ▶ Scientific papers (arXiv, PubMed)
- ▶ Code repositories (GitHub)
- ▶ Dialogue data (for conversational ability)

Data Volume

- ▶ Typically 1-10 trillion tokens
- ▶ GPT-3: 300B tokens
- ▶ LLaMA 2: 2T tokens; LLaMA 3: 15T tokens;
- ▶ Deepseek V3: 14.8T tokens
- ▶ Qwen 3: 36T tokens

Step 2: Data Preprocessing

Cleaning Steps

- ▶ Deduplication
- ▶ Quality filtering
- ▶ Toxicity removal
- ▶ Language identification
- ▶ PII redaction

Data quality is very important.

Tokenization

- ▶ Subword algorithms:
 - ▶ BPE (GPT series)
 - ▶ WordPiece (BERT)
 - ▶ Unigram (SentencePiece)
- ▶ Vocabulary size: 32K-200K

Step 3: Model Architecture

Transformer Specifications

- ▶ Decoder-only architecture (GPT-style)
- ▶ Key hyperparameters:
 - ▶ Layers: 12-96
 - ▶ Hidden dim: 768-12288
 - ▶ Attention heads: 12-128
 - ▶ Context window: 2K-32K tokens

Example Configurations

Model	Layers	Dim	Params
GPT-3	96	12288	175B
LLaMA 2	80	8192	70B
PaLM	118	18432	540B

	8B	70B	405B
Layers	32	80	126
Model Dimension	4,096	8192	16,384
FFN Dimension	14,336	28,672	53,248
Attention Heads	32	64	128
Key/Value Heads	8	8	8
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Activation Function	SwiGLU		
Vocabulary Size	128,000		
Positional Embeddings	RoPE ($\theta = 500,000$)		

Figure: Architecture for LLaMa3

Step 4: Training Objectives

Primary Objective

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P(x_t | x_{<t}; \theta)$$

Autoregressive language modeling (next token prediction)

Common Variants

- ▶ Causal masking (left-to-right)
- ▶ Fill-in-the-middle (for code models)
- ▶ Mixed objective (sometimes with span corruption)

Step 5: Optimization

Key Techniques

- ▶ AdamW optimizer
- ▶ Learning rate warmup
- ▶ Cosine decay schedule
- ▶ Gradient clipping
- ▶ Mixed precision training

Challenges

- ▶ Batch size: 1M-10M tokens
- ▶ Hardware: 100s-1000s GPUs/TPUs
- ▶ Training time: Weeks-months
- ▶ Memory optimization:
 - ▶ ZeRO
 - ▶ Pipeline parallelism
 - ▶ Tensor parallelism

Step 6: Evaluation & Scaling

Training Monitoring

- ▶ Loss curves
- ▶ Perplexity
- ▶ Downstream task performance
- ▶ Zero-shot capabilities

Post-Training: The Alignment Phase

Three Key Stages

1. Supervised Fine-Tuning (SFT)

- ▶ Trains on curated (input, output) pairs
- ▶ Adapts base model to follow instructions
- ▶ Requires 10K-100K high-quality examples

2. Reinforcement Learning from Human Feedback (RLHF)

- ▶ Learns from preference rankings (good vs bad outputs)
- ▶ Uses reward model trained on human judgments
- ▶ Optimizes with PPO algorithm

3. Constitutional AI

- ▶ Applies self-critique against principles
- ▶ Reduces harmful outputs without human labels
- ▶ Uses chain-of-thought feedback

Two Pillars of Modern Deep Learning

Scaling Laws

- ▶ **Model size, dataset size, and compute** scale predictably
- ▶ Performance follows power-law relationships
- ▶ Enables systematic improvement planning
- ▶ Guides architecture and resource allocation

GPU Acceleration

- ▶ **Massive parallelism** for matrix operations
- ▶ 100-1000 $\times$ speedup over CPUs
- ▶ Essential for training large models
- ▶ Enables practical training times

Key Insight

Scaling laws tell us **what** is possible; GPU acceleration makes it **practically achievable**

Scaling laws

Scaling Laws

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

- ▶ N : Model parameters
- ▶ D : Training tokens
- ▶ α_N, α_D : Scaling exponents
- ▶ L is the error term

The LLM Revolution

- ▶ **Observation:** Larger models perform better
- ▶ **Question:** How does performance scale with resources?
- ▶ **Key Insight:** Smooth, predictable relationships exist

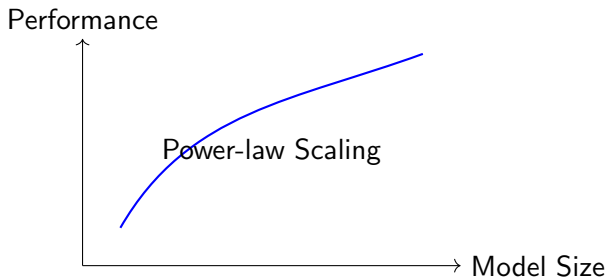


Figure: The scaling phenomenon

The Fundamental Power Law

Kaplan et al. (2020) Formula

$$L(N, D) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

- ▶ L : Test loss (cross-entropy)
- ▶ N : Number of parameters
- ▶ D : Number of training tokens
- ▶ $N_c, D_c, \alpha_N, \alpha_D$: Fitted constants

Compute Budget

$$C \approx 6ND \quad (\text{Training FLOPs})$$

Three Scaling Regimes

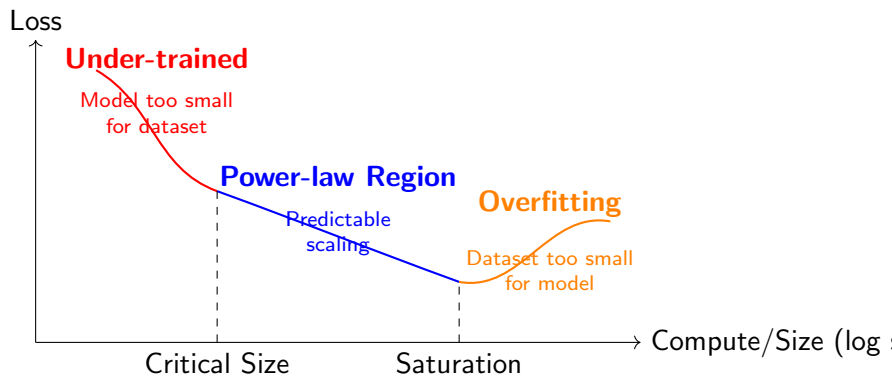


Figure: The three scaling regimes

The Chinchilla Revolution

Before Chinchilla:

Make models bigger

$$D \propto C^{0.74}$$

$$N \propto C^{0.73}$$

After Chinchilla:

"Balance is key"

$$D_{opt} \propto C^{0.50}$$

$$N_{opt} \propto C^{0.50}$$

Model	Parameters
Gopher	280B
blue!10 Chinchilla	70B
GPT-3	175B

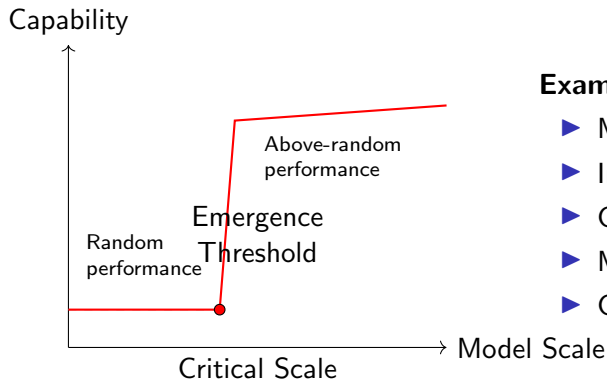
Table: Chinchilla outperformed larger models

Compute-Optimal Formula

$$N_{opt} = 1.95 \times 10^8 \times C^{0.50}$$

$$D_{opt} = 2.21 \times 10^4 \times C^{0.50}$$

The Emergence Phenomenon



Examples:

- ▶ Multi-step reasoning
- ▶ Instruction following
- ▶ Code generation
- ▶ Mathematical proofs
- ▶ Cross-lingual transfer

Figure: Emergent abilities appear suddenly

Key Insight

Emergence suggests scaling isn't just quantitative improvement

CPU vs GPU Architecture

CPU

- ▶ Few powerful cores (2-16)
- ▶ Optimized for sequential tasks
- ▶ Large cache, low latency
- ▶ Complex control logic

GPU

- ▶ Thousands of simple cores (1000-10,000+)
- ▶ Optimized for parallel tasks
- ▶ High bandwidth, higher latency
- ▶ Simple control logic

Matrix Multiplication is Embarrassingly Parallel

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

- ▶ Each output element C_{ij} can be computed **independently**
- ▶ Perfect for massive parallelization
- ▶ Thousands of threads can work simultaneously

Key Insight

GPUs can launch one thread per output element!

GPU Thread Hierarchy for Matrix Multiplication

- ▶ **Grid:** Organizes all thread blocks
- ▶ **Block:** Group of threads (e.g., $16 \times 16 = 256$ threads)
- ▶ **Thread:** Computes one output element

Performance Comparison

Matrix Size	CPU Time	GPU Time
1024×1024	250 ms	5 ms
2048×2048	2.0 s	25 ms
4096×4096	16 s	150 ms
8192×8192	128 s	1.2 s

Typical Speedups

- ▶ Small matrices: 10-50 $\times$ faster
- ▶ Large matrices: 100-1000 $\times$ faster

Why Deep Learning Loves GPUs

Neural Network Operations

- ▶ Fully Connected: Matrix multiplication
- ▶ Convolutions: Matrix multiplication (im2col)
- ▶ Attention: Large matrix multiplications
- ▶ Batch processing: More parallelism

Specialized Hardware

- ▶ Tensor Cores (NVIDIA)
- ▶ Matrix Cores (AMD)
- ▶ Mixed-precision operations
- ▶ Dedicated matrix units

Summary

Why GPUs Accelerate Matrix Multiplication

- ▶ **Massive parallelism:** Thousands of independent computations
- ▶ **Memory optimization:** Shared memory and coalesced access
- ▶ **Specialized hardware:** Tensor cores and matrix units
- ▶ **Data reuse:** Tiling minimizes global memory access

Result

100-1000 $\times$ speedup for large matrix operations
This is why deep learning runs on GPUs!

Large efforts are needed for training on multi-gpu, multi-node.

Section 4: Graph neural network

The Limitation of Traditional Neural Networks

The Problem

Traditional neural networks assume **grid-like** or **sequential** data:

- ▶ CNNs: Images (regular grids)
- ▶ RNNs: Text/Speech (sequences)

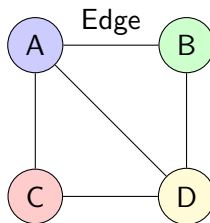
But Real-World Data is Often Relational!

- ▶ Social networks
- ▶ Molecular structures
- ▶ Citation networks
- ▶ Transportation systems

Graph Fundamentals

Graph Components

- ▶ **Nodes/Vertices:** Entities
- ▶ **Edges/Links:** Relationships
- ▶ **Node Features:** Attributes
- ▶ **Edge Features:**
Relationship properties



Core Idea: Learning from Relationships

”A node is defined by its neighbors”

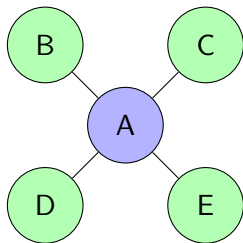
Key Insight

GNNs learn node representations by **aggregating information** from neighboring nodes

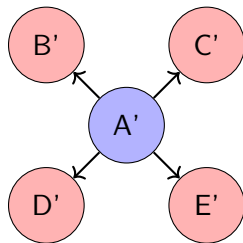
Real-world Example

Predicting paper influence: Content matters, but **citation relationships** matter more!

Message Passing: How GNNs Work



Layer 0: Initial Features



Layer 1: Knows Neighbors

Message Passing Algorithm

Three Key Steps

1. **Message:** Each node creates a message for its neighbors
2. **Aggregation:** Collect messages from all neighbors
3. **Update:** Combine own state with aggregated messages

$$h_v^{(l+1)} = \text{UPDATE} \left(h_v^{(l)}, \text{AGGREGATE} \left(\{h_u^{(l)}, \forall u \in \mathcal{N}(v)\} \right) \right)$$

- ▶ $h_v^{(l)}$: Representation of node v at layer l
- ▶ $\mathcal{N}(v)$: Neighbors of node v
- ▶ UPDATE, AGGREGATE: Learnable functions

Graph Convolutional Networks (GCN)

Key Idea

Normalized aggregation of neighbor features

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right)$$

- ▶ $\tilde{A}$: Adjacency matrix with self-loops
- ▶ $\tilde{D}$: Degree matrix
- ▶ $H^{(l)}$: Node features at layer l
- ▶ $W^{(l)}$: Learnable weights

Graph Attention Networks (GAT)

Key Innovation

Attention mechanism - not all neighbors are equally important!

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [Wh_i || Wh_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(a^T [Wh_i || Wh_k]))}$$

Other GNN Variants

GraphSAGE

- ▶ Samples fixed-size neighborhoods
- ▶ Scalable to large graphs
- ▶ Inductive learning

Graph Autoencoders

- ▶ Unsupervised learning
- ▶ Graph generation
- ▶ Anomaly detection

Graph Isomorphism Networks

- ▶ Powerful theoretical foundation
- ▶ Maximum expressive power
- ▶ Sum aggregation + MLP

Spatial-Temporal GNNs

- ▶ Dynamic graphs
- ▶ Time-series on graphs
- ▶ Traffic prediction

Real-World Applications

Science & Medicine

- ▶ **Molecule property prediction**
- ▶ Drug discovery
- ▶ Protein interface prediction
- ▶ Disease spread modeling

Technology

- ▶ **Recommendation systems**
- ▶ Fraud detection
- ▶ Knowledge graphs
- ▶ Program analysis

Industry

- ▶ **Social network analysis**
- ▶ Traffic prediction
- ▶ Supply chain optimization
- ▶ Financial risk assessment

Computer Vision

- ▶ Scene graph generation
- ▶ Point cloud processing
- ▶ Human pose estimation

Current Challenges

Technical Challenges

- ▶ Scalability to large graphs
- ▶ Over-smoothing with deep networks
- ▶ Dynamic graph learning
- ▶ Heterogeneous graphs

Theoretical Challenges

- ▶ Expressive power limits
- ▶ Generalization guarantees
- ▶ Interpretability
- ▶ Robustness to adversarial attacks

Future Directions

Emerging Research Areas

- ▶ **Graph Transformers:** Self-attention on graphs
- ▶ **Self-supervised Learning:** Pre-training on graphs
- ▶ **Explainable GNNs:** Understanding predictions
- ▶ **Graph Foundation Models:** Large-scale pre-training
- ▶ **Geometric Deep Learning:** Unified theory

Summary

Key Takeaways

- ▶ GNNs learn from **relational data** through message passing
- ▶ Core operation: **Aggregate** → **Combine** → **Update**
- ▶ Multiple architectures: GCN, GAT, GraphSAGE, etc.
- ▶ Wide range of applications across domains
- ▶ Active research area with many open challenges

Graphs are everywhere - GNNs help us learn from them!